Marathi Hate Speech Detection Using Domain Adapted MuRIL

Submitted by:

RUTVIK MAKARAND GOKHALE

261090830002

M.Tech. in Artificial Intelligence and Decision Science

School of Computer Engineering



MANIPAL INSTITUTE OF TECHNOLOGY

MANIPAL

(A constituent unit of MAHE, Manipal)

October 30, 2026

Abstract

With the rapid growth of social media, there is an increase in the amount of offensive and hateful content online. However, due to the lack of annotated datasets and NLP specific resources for Marathi hate speech detection remains a challenge. In this project, I have created a Marathi hate speech detection system using the transformer model MuRIL, with Domain Adaptive Pretraining.

MuRIL (google/muril-base-cased) was pretrained on a random sample of 20,000 sentences from the MeCorpus Marathi corpus. With Masked Language Modelling objective for 1 epoch. This model was then fine-tuned for 3 epochs using the MahaHate binary dataset. This dataset contains 37,500 comments; each marked as HOF or NOT. The data was also pooled and split into 80/20 stratified training and test sets.

To compare how much the model improved with the domain adaptation, I also fine-tuned the original MuRIL. The parameters used were 3 epochs, learning rate of 2 10-5 and effective batch size of 32. These metrics were calculated on 7500 comments in the held-out set. The base MuRIL achieved an accuracy and F1 score of 0.9095, and ROC AUC score of 0.9691. The DAPT model achieved an accuracy of 0.9153, F1 score of 0.9153, and ROC AUC score of 0.9723. The loss decreased from 0.532 to 0.489, and there was a reduction of 44 misclassified comments. However, this is a small improvement that only came from running once, so the improvement can be considered as indicative.

The whole pipeline runs in one Kaggle notebook. There were some limitations in this project such as the short domain adaptation period (due to the high loss), using the test set for monitoring during fine-tuning, and no text cleaning was done.

Contents

Abstract i

1 Introduction 1

2 Problem Definition 2
2.1 Growth of Harmful Content on Social Media 2
2.2 Challenges in Marathi Hate Speech Detection 2
2.3 Need for Automated Moderation 2
2.4 Domain Adaptation Requirement 2
Problem Statement 2

3 Scope of Problem 3
3.1 In Scope 3
3.2 Out of Scope 3

4 Motivation 4

5 Objectives 5

6 Methodology and Implementation 6
6.1 Datasets 6
6.2 Data Preparation 7
6.3 Marathi Corpus Creation 8
6.4 MuRIL Tokenization 8
6.5 Domain Adaptive Pretraining 8
6.6 Fine-Tuning 9
6.7 Model Evaluation 10
6.8 Implementation Environment 10

7 Hardware and Software Requirements 11
7.1 Hardware Requirements 11
7.2 Software & Libraries 11

8 Timeline 12

9 Results and Evaluation 13
9.1 Training Performance 13
9.1.1 Domain Adaptive Pretraining 13
9.1.2 Fine-Tuning 13
9.2 Classification Metrics 15

9.3 Confusion Matrix 16
9.4 Classification Report 17
9.5 ROC Curve 18
9.6 Comparison: Base MuRIL vs DAPT-MuRIL 18
9.7 Qualitative Test on Sample Sentences 19
9.8 Computational Cost 19
9.9 Discussion 19

10 Conclusion and Future Work **21**
10.1 Conclusion 21
10.2 Future Work 21

References **22**

List of Figures

6.1 Overall pipeline of the system 6
6.2 Label distribution of the MahaHate dataset (0 = NOT, 1 = HOF) 7

9.1 Training loss of both models during fine-tuning 14
9.2 Validation accuracy per epoch 14
9.3 Validation F1-score per epoch 15
9.4 Accuracy and F1-score of MuRIL and DAPT-MuRIL 16
9.5 Confusion matrix of base MuRIL (0 = NOT, 1 = HOF) 16
9.6 Confusion matrix of DAPT-MuRIL (0 = NOT, 1 = HOF) 17
9.7 ROC curves of both models (AUC 0.9691 and 0.9723) 18

List of Tables

6.1 Datasets used and their row counts 7
6.2 MahaHate split used for fine-tuning and evaluation 8
6.3 DAPT configuration 9
6.4 Fine-tuning configuration of the two models 10

7.1 Software and libraries used 11

8.1 10-Week Project Timeline with Milestone Outcomes 12

9.1 Masked language modeling loss during DAPT (values from the training log) 13
9.2 Per-epoch results during fine-tuning (evaluation is on the test set) 14
9.3 Classification metrics of the final models on the test set 15
9.4 Confusion matrix counts (positive class = HOF) 16
9.5 Per-class results derived from the confusion matrices (they agree with the rounded scikit-learn reports in the notebook) 17
9.6 Comparison of base MuRIL and DAPT-MuRIL under identical fine-tuning settings 18
9.7 Predictions on four hand-written sample sentences 19
9.8 Run time of each stage on the dual-GPU Kaggle runtime 19

Chapter 1

Introduction

Social media platforms contain a vast amount of user generated content. Many of these contain offensive, abusive, or hateful content. This content is often directed at individuals or groups based on religion, caste, ethnicity, gender, or nationality. It can cause issues in social and psychological circles. To avoid going through this content manually, detecting hate speech has become an important application of NLP.

Many improvements have been made on English and other high resource languages however; Marathi is a low resource language. Due to a lack of annotated datasets and pre-trained resources, in addition to it having complex Morphology, Devanagari script, and informal spelling on social media.

In this project a Marathi hate speech detection model has been created using the MuRIL (Multi-lingual Representations for Indian Languages) [ ] model. This model is a BERT based model which has been pre-trained on Indian languages. As the model doesn't cover much Marathi text, a Domain Adaptive Pre-training technique [ ] was used. This involved using a sample of Marathi text from the MeCorpus corpus to continue training the model on a Masked Language Model objective. The model was then fine-tuned using the MahaHate dataset [ ] to classify if the comments would be either hateful/offensive or not.

I have created a controlled experiment. The original version of MuRIL (google/muril-base-cased), will be fine-tuned using the same settings. The DAPT model will be using the original MuRIL as its starting checkpoint. This model has been adapted for 1 epoch on 20,000 sentences from the MeCorpus. The models will then be fine-tuned on the MahaHate dataset. This will allow me to compare the models to see how much improvement was made during the domain adaptation stage.

The final model will be built as a reproducible Python notebook on Kaggle. This will include loading, sampling, tokenizing, adapting, fine-tuning both models, and evaluating them. This project will incorporate transfer learning, domain adaptation, and standard classification metrics for a low resource language.

Chapter 2

Problem Definition

2.1 Increase of Harmful Content on Social Media

Due to the increase in user-generated content, there has been an increase in offensive and hateful content on social media. This can cause animosity between different communities, and can be very harassing to individuals.

2.2 Issues with in Marathi Hate Speech Detection

Marathi is a low resource language. This means there aren't many annotated datasets or pre-trained resources to use. Hate speech is often said in a sarcastic, slang, and implicit way. This means the text often includes many keywords that cannot be captured with a keyword list. Additionally, Marathi often contains many Hindi and English words.

2.3 Automated Moderation Required

Manually moderating the content is a slow, expensive and emotionally draining process. It is not feasible to go through all of the content. Automated systems can be built to automatically moderate content, and highlight likely harmful content.

2.4 Domain Adaptation Required

General multi-lingual models may not capture all the features of Marathi. By continuing to pre-train the model on Marathi text, we expect to get better representations of the text.

Problem Statement

An automated Marathi hate speech detection system is required that can:

(i) Process Marathi text data,

(ii) Learn Marathi linguistic characteristics through domain adaptation,

(iii) Identify if the content is hateful/offensive or no,

(iv) Perform well in classifications, and

(v) Help with social media moderation.

Chapter 3

Scope of Problem

A text based hate speech classifier has been created using domain adapted MuRIL for Marathi. This project does not cover;

3.1 In Scope

- Binary Marathi text classification (NOT versus HOF).
- Load, sample and tokenize Marathi text.
- Domain Adaptive Pretraining of MuRIL on a sample of the MeCorpus corpus.
- Fine-tune both models (base MuRIL and adapted) using MahaHate.
- Compare base MuRIL and DAPT MuRIL.
- Calculate metrics (accuracy, precision, recall, f1).
- Create confusion matrix, classification report and ROC curve.
- Small qualitative analysis of handmade sentences.
- A reproducible Kaggle notebook and saved model checkpoints.

3.2 Out of Scope

- Custom text cleaning, normalising and removing duplicates
- Integration with social media to process real time content
- Detect hate speech in images and videos
- Detect hate speech in multiple languages
- Explainable AI techniques
- Using context or user metadata
- Multiple runs using several random seeds and significance testing.

Chapter 4

Motivation

Hate speech on social media causes serious social issues. It can be difficult to detect manually due to the large amount of communication online. While modern NLP techniques have produced great results for English, Marathi hasn't been explored much.

The motivation for this project is fivefold:

1. Automated Ensure safer communication online. Automating detection will help platforms and moderators identify harmful content quicker.

2. Help develop NLP for Marathi. Marathi has a large population of speakers, but lacks tools and resources.

3. Pretrained transformers can be effectively used for low resource languages.

4. Domain Adaptation can be beneficial. As there is no labelled data, using unlabelled data is far easier to obtain. This may help improve the pre-trained model cheaply.

5. Help with content moderation using the classifier.

Chapter 5

Objectives

The main objectives of the project are:

1. Collect datasets containing Marathi (MeCorpus, Dakshina and MahaHate).
2. Prepare a corpus for Marathi adaptation by sampling and removing null values. Prepare the MahaHate labels and splits.
3. Domain Adaptive Pretraining using masked language modelling on MuRIL.
4. Fine-tune the adapted model using MahaHate.
5. Calculate metrics (accuracy, precision, recall, f1).
6. Fine-tune base MuRIL using same settings as adapted model. Compare models.
7. Create a reproducible pipeline in a Kaggle notebook and save the trained model.
8. Document the performance and limitations of the model.

Chapter 6

Methodology and Implementation

The model consists of 2 sections which merge during fine-tuning (see Figure 6.1). The MeCorpus unlabelled text was sampled. This was used to adapt the MuRIL model using masked language modelling. The labelled MahaHate data was combined, encoded and split. The adapted model was then fine-tuned on this data and evaluated. All of this was completed in one Kaggle notebook.

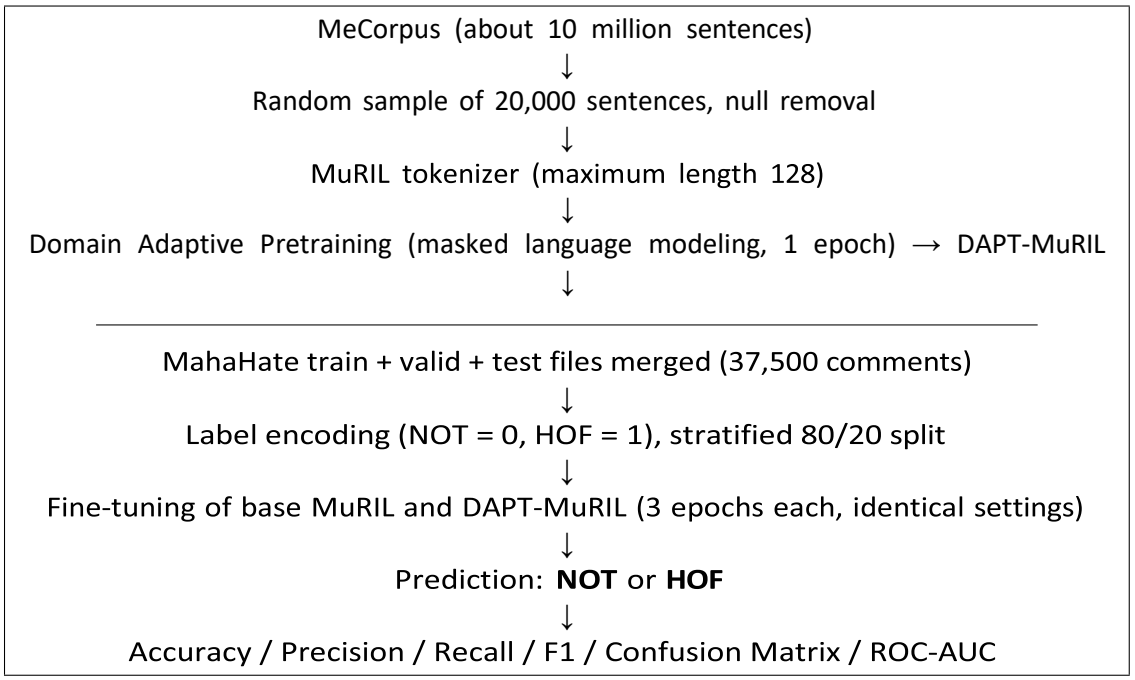


Figure 6.1: Overall pipeline of the system

6.1 Datasets

The MeCorpus dataset contains a large amount of Marathi text. The file I used had ~10 million rows. I loaded the full dataset (9,999,882 rows). This was used as the unlabelled data.

The MahaHate dataset (L3Cube-MahaHate) is a human annotated Marathi hate speech dataset based on tweets []. The binary version was used. This data was split into training, validation and test sets. The labels were either NOT (not offensive) or HOF (hateful/offensive). This was the only labelled dataset. I combined the three files into one table containing 37,500 comments.

The Dakshina dataset [] contains text in 12 South Asian languages including Marathi. This was considered as additional Marathi text, but wasn't used in the final experiments.

Dataset	Role in final run	Labeled	Rows
MeCorpus	Source of DAPT text	No	9,999,882
Sampled for DAPT	Adaptation corpus	No	20,000
MahaHate	Fine-tuning and evaluation	Yes	37,500

Table 6.1: Datasets used and their row counts

The MahaHate labels are balanced: 18,750 comments in each class (Figure 6.2). Because of this, no resampling or class weighting was needed.

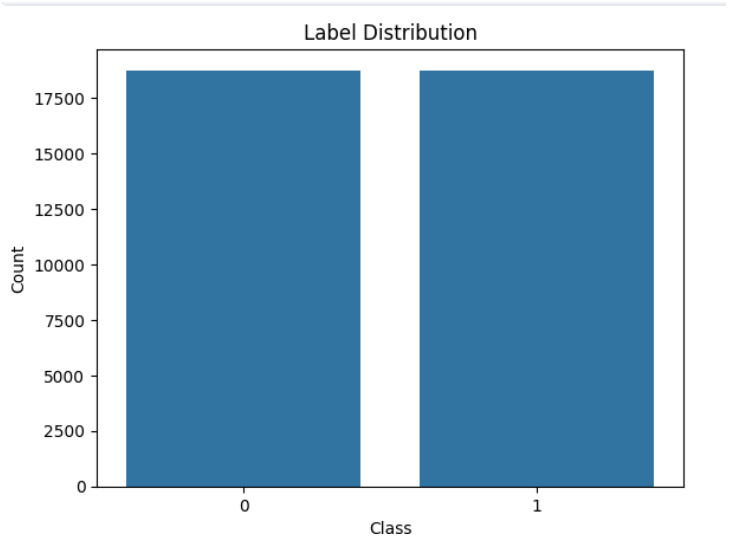


Figure 6.2: Label distribution of the MahaHate dataset (0 = NOT, 1 = HOF)

6.2 Data Preparation

The following preparation steps are performed in the notebook:

- **Sampling** - 20,000 rows were randomly sampled from the MeCorpus (random seed = 42). Any rows with missing text were dropped. No rows were removed as the 20,000 sentences provided exactly 625 training steps with the effective batch size of 32.
- **Encoding** - NOT = 0, HOF = 1.
- **Train/Test split** - The MahaHate data was split into 80/20 using stratified sampling based on the label (random seed = 42). This resulted in 30,000 training comments and 7,500 test comments (see Table).

No custom text cleaning, Unicode normalisation, emoji handling or duplicates were done. Data was passed directly to the tokenizer. It wasn't checked if there were any duplicates in the training and test data. This was a limitation.

Split	Total	NOT (0)	HOF (1)
Training	30,000	15,000	15,000
Test	7,500	3,750	3,750

Table 6.2: MahaHate split used for fine-tuning and evaluation

There is no separate validation set. The original validation file was merged into the pool before splitting, and the per-epoch evaluations during fine-tuning use the test set (Section 6.6).

6.3 Marathi Corpus Creation

The adaptation corpus is the 20,000-sentence random sample of MeCorpus described above. It is held as a Hugging Face Dataset with a single text column. The sample is small compared with the full corpus, so that adaptation could finish within the available notebook time (about 11 minutes).

6.4 MuRIL Tokenization

Text is converted with the MuRIL tokenizer, a WordPiece tokenizer with a large vocabulary covering Indian languages. For adaptation, each sentence is truncated to a maximum of 128 tokens, and padding is added per batch by the data collator. For fine-tuning, texts are truncated at the tokenizer's default maximum length and padded to the longest sequence, and the tokenizer saved with the adapted model is used. Subword tokenization lets the model represent rare or unseen Marathi words from known pieces.

6.5 Domain Adaptive Pretraining

The base model is the pretrained MuRIL encoder google/muril-base-cased (BERT-base architecture with 12 layers, hidden size 768 and 12 attention heads), loaded through Hugging Face Transformers [1, 8]. Its self-attention is

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^{\top}}{\sqrt{d_k}}\right) V \tag{6.1}$$

as introduced in the Transformer [5].

The model is loaded with a masked language modeling head. During adaptation 15% of the tokens in each batch are selected for masking by the data collator, with the usual replacement by [MASK], a random token or no change [4], and the model predicts the original tokens by minimizing

$$L_{\text{MLM}} = -\sum_{i \in M} \log P(x_i \mid \tilde{x}) \tag{6.2}$$

where M is the set of masked positions and $\tilde{x}$ is the corrupted input. The objective is self-supervised, so no labels are needed. Training uses the Hugging Face Trainer with its

default AdamW optimizer [7] and linear learning-rate decay. The run was on two GPUs, so the effective batch size was 32 (16 per device) and one epoch took 625 steps. The result was saved as the domain-adapted checkpoint, called **DAPT-MuRIL**.

Hyperparameter	Value
Base model	google/muril-base-cased
Objective	Masked language modeling
Masking probability	0.15
Adaptation corpus	20,000 sentences (MeCorpus sample)
Maximum sequence length	128
Batch size	16 per device (32 effective, 2 GPUs)
Learning rate	5e-5 (Trainer default)
Epochs	1 (625 steps)
Optimizer	AdamW
Logging interval	every 100 steps
Training time	668 s (about 11 min)

Table 6.3: DAPT configuration

6.6 Fine-Tuning

Two classifiers are trained, one from each starting checkpoint, with exactly the same procedure. Each is loaded for sequence classification with two output labels. The classification head consists of the pooler layer, dropout and a linear layer on the [CLS] representation. The linear layer is always newly initialized. The pooler is loaded from the checkpoint for base MuRIL, but it is newly initialized for DAPT-MuRIL, because the masked language modeling checkpoint does not contain it. This difference is a disadvantage for the adapted model. The whole model is trained on the 30,000 training comments with cross-entropy loss:

$$L_{\mathrm{CE}} = -\frac{1}{N}\sum_{n=1}^{N}\sum_{c} y_{n,c}\log \hat{y}_{n,c} \tag{6.3}$$

At the end of each epoch the model is evaluated on the 7,500-comment test set, and accuracy and weighted F1-score are logged. No early stopping or best-checkpoint selection is used, so the model after epoch 3 is the one reported for each arm. Since the evaluation set is also the test set, the per-epoch curves in Chapter 9 are test-set measurements, called validation curves in the figures because the Trainer reports them as validation.

Hyperparameter	Base MuRIL	DAPT-MuRIL
Starting checkpoint	google/muril-base-cased	Adapted checkpoint
Task	Binary classification (NOT / HOF)	
Training / test comments	30,000 / 7,500	
Maximum sequence length	Tokenizer default (padded to longest)	
Batch size	16 per device (32 effective, 2 GPUs)	
Evaluation batch size	16 per device	
Learning rate	2e-5	
Epochs	3 (2,814 steps)	
Optimizer	AdamW, linear decay, no warmup	
Loss	Cross-entropy	
Pooler layer	Pretrained	Newly initialized
Training time	3,536 s (about 59 min)	3,548 s (about 59 min)

Table 6.4: Fine-tuning configuration of the two models

6.7 Model Evaluation

The classifier is evaluated on the 7,500 held-out comments. With TP, TN, FP and FN denoting true positives, true negatives, false positives and false negatives, the metrics are

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \qquad \text{Precision} = \frac{TP}{TP + FP}, \tag{6.4}$$

$$\text{Recall} = \frac{TP}{TP + FN}, \qquad \text{F1} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \tag{6.5}$$

The Trainer reports accuracy and the support-weighted F1-score; a per-class classification report, a confusion matrix and the ROC curve with its area (AUC) are produced with scikit-learn [10]. As the test set has equal class sizes, macro and weighted averages are identical.

6.8 Implementation Environment

The pipeline is written in Python on a Kaggle notebook with a dual-GPU runtime (NVIDIA T4 ×2). PyTorch [9] is the deep learning backend, Hugging Face Transformers, Datasets and Evaluate handle models, tokenization and metrics [8], and Pandas, NumPy, scikit-learn, Matplotlib and Seaborn handle data, evaluation and plots. Random seeds for sampling and splitting are set to 42. The adapted checkpoint is saved to the notebook's working directory, and the training histories of both classifiers are written to CSV files.

Chapter 7

Hardware and Software Requirements

7.1 Hardware Requirements

- Kaggle notebook runtime with GPU acceleration (two NVIDIA T4 GPUs were used).
- Enough RAM to load the roughly 10-million-row MeCorpus file (provided by the Kaggle runtime).
- Internet connectivity to download the MuRIL model from the Hugging Face hub.

7.2 Software & Libraries

Purpose	Tool
Programming environment	Python 3.13, Kaggle Notebooks
Deep learning	PyTorch
Models and tokenization	Hugging Face Transformers, SentencePiece, Accelerate; MuRIL (google/muril-base-cased)
Dataset handling	Hugging Face Datasets, Pandas, NumPy
Metrics	Evaluate, scikit-learn
Visualization	Matplotlib, Seaborn

Table 7.1: Software and libraries used

Chapter 8

Timeline

The project followed a phased plan from dataset collection to documentation.

Week	Phase	Focus / Deliverable
1	Phase 1	Dataset collection: MeCorpus, Dakshina and MahaHate.
2	Phase 2	Data analysis: dataset sizes and class distribution.
3	Phase 3	Data preparation: label encoding, splitting and null removal.
4	Phase 4	Adaptation corpus creation: sampling from MeCorpus.
5	Phase 5	MuRIL loading: model and tokenizer setup and verification.
6	Phase 6	Domain Adaptive Pretraining with masked language modeling.
7	Phase 7	Fine-tuning of base MuRIL and DAPT-MuRIL on MahaHate.
8	Phase 8	Evaluation on the test set.
9	Phase 9	Performance analysis: comparison of the two models, curves, confusion matrices, ROC and sample predictions.
10	Phase 10	Documentation and presentation.

Table 8.1: 10-Week Project Timeline with Milestone Outcomes

Chapter 9

Results and Evaluation

All classification results below are for the final epoch-3 models, evaluated on the held-out MahaHate test set of 7,500 comments, 3,750 NOT (class 0) and 3,750 HOF (class 1). Both the base MuRIL classifier and the DAPT-MuRIL classifier were trained under identical settings. Values read from plots are approximate.

9.1 Training Performance

9.1.1 Domain Adaptive Pretraining

The masked language modeling loss during adaptation was logged every 100 steps (Table 9.1). It fell from 10.26 at step 100 to 8.89 at step 600, and the average over the 625-step epoch was 9.36. The loss was still decreasing when training stopped, apart from a small rise between steps 400 and 500, and it remained high in absolute terms, so the adaptation was short and far from converged.

Step	Training loss
100	10.2599
200	9.6822
300	9.4246
400	9.0206
500	9.0377
600	8.8929
Average (625 steps)	9.3570

Table 9.1: Masked language modeling loss during DAPT (values from the training log)

9.1.2 Fine-Tuning

Table 9.2 gives the per-epoch results of both models. The training loss is the last value logged before each evaluation (the loss is logged every 500 steps). It falls steadily in both models (Figure 9.1). The evaluation loss of base MuRIL does not improve after epoch 1, whereas that of DAPT-MuRIL keeps falling and ends lower (Table 9.2).

Model	Epoch	Train loss	Eval loss	Accuracy	F1
MuRIL	1	0.9481	0.5240	0.8987	0.8987
	2	0.4554	0.5486	0.9100	0.9100
	3	0.3128	0.5319	0.9095	0.9095
DAPT-MuRIL	1	0.9337	0.5172	0.9036	0.9036
	2	0.4602	0.4972	0.9132	0.9132
	3	0.3084	0.4894	0.9153	0.9153

Table 9.2: Per-epoch results during fine-tuning (evaluation is on the test set)

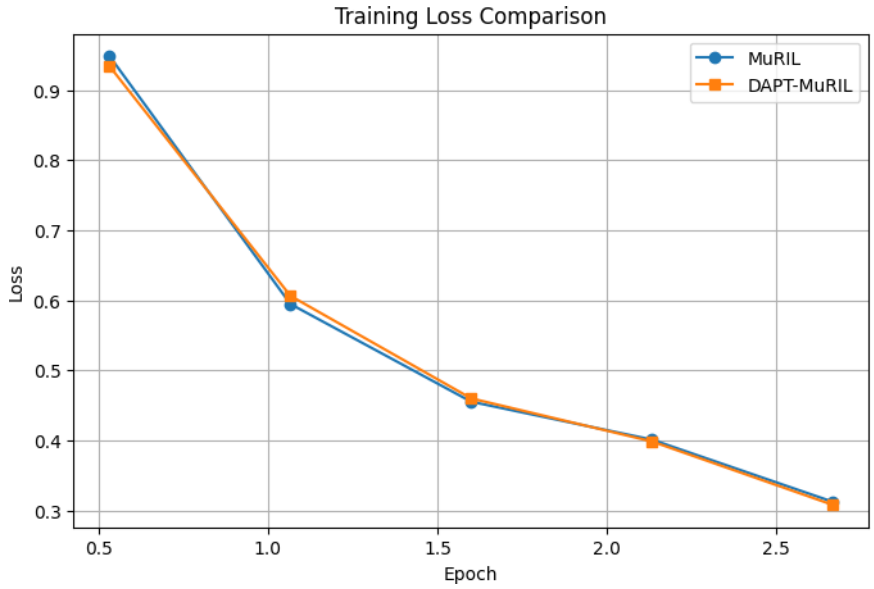


Figure 9.1: Training loss of both models during fine-tuning

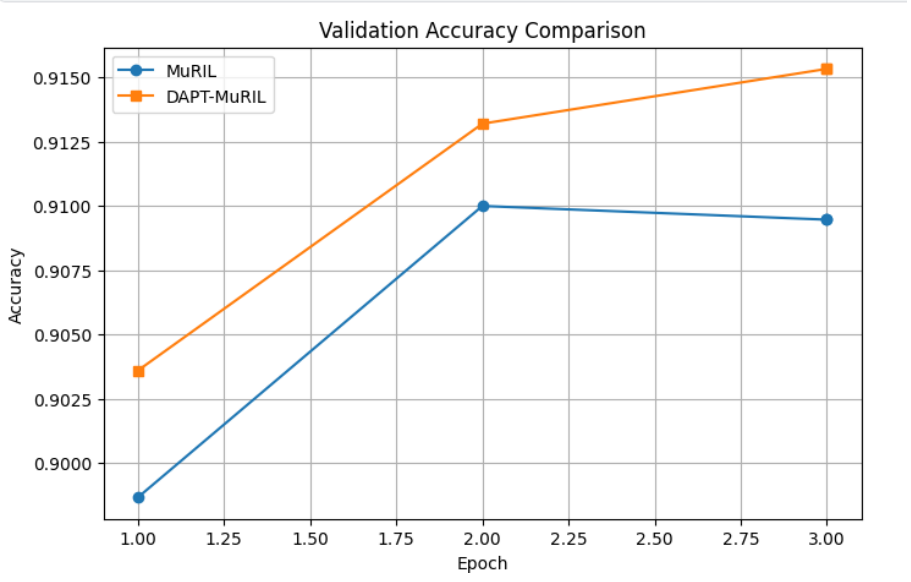


Figure 9.2: Validation accuracy per epoch

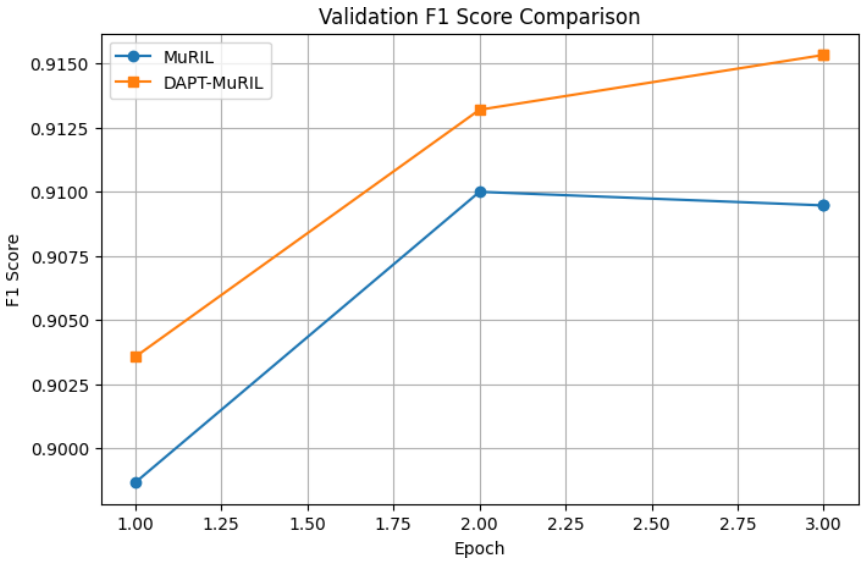


Figure 9.3: Validation F1-score per epoch

The adapted model is ahead after every epoch (accuracy 0.9036 against 0.8987 after epoch 1, 0.9132 against 0.9100 after epoch 2, and 0.9153 against 0.9095 after epoch 3). The accuracy of base MuRIL dips slightly between epochs 2 and 3, while that of DAPT-MuRIL keeps rising (Figures 9.2 and 9.3). Accuracy and F1 are almost identical in every row because the classes are balanced.

9.2 Classification Metrics

Model	Accuracy	Precision	Recall	F1	Eval loss	ROC-AUC
MuRIL	0.9095	0.9095	0.9095	0.9095	0.5319	0.9691
DAPT-MuRIL	0.9153	0.9153	0.9153	0.9153	0.4894	0.9723

Table 9.3: Classification metrics of the final models on the test set

Accuracy, the weighted F1-score and the evaluation loss are from the Trainer evaluation. Precision and recall are macro averages computed from the confusion matrices, and because the two test classes are equal in size they coincide with the weighted averages.

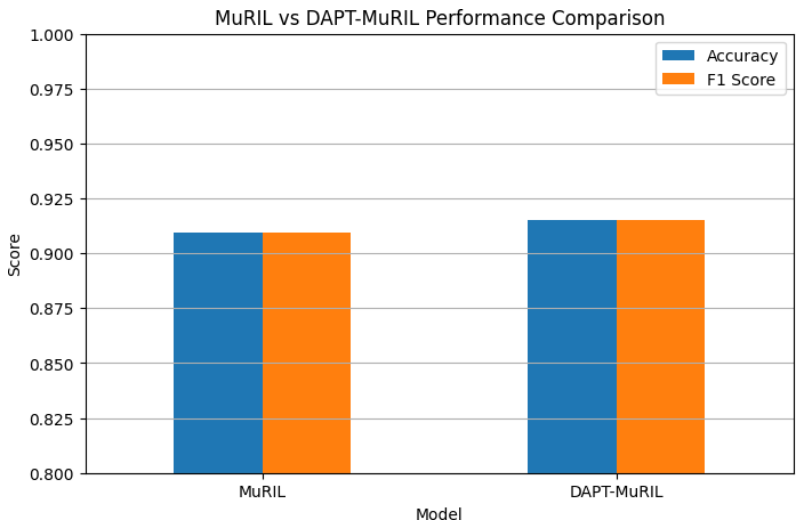


Figure 9.4: Accuracy and F1-score of MuRIL and DAPT-MuRIL

9.3 Confusion Matrix

Model	TN	FP	FN	TP	Errors	Error rate
MuRIL	3,430	320	359	3,391	679	9.05%
DAPT-MuRIL	3,432	318	317	3,433	635	8.47%

Table 9.4: Confusion matrix counts (positive class = HOF)

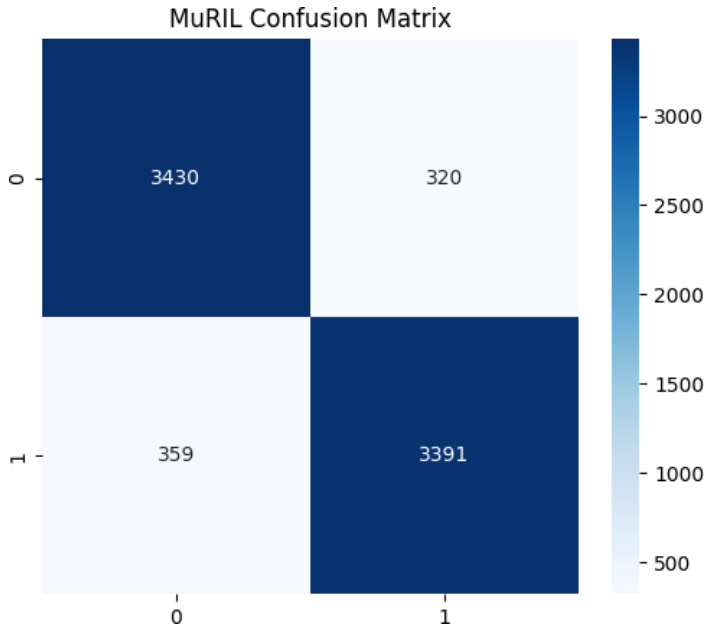


Figure 9.5: Confusion matrix of base MuRIL (0 = NOT, 1 = HOF)

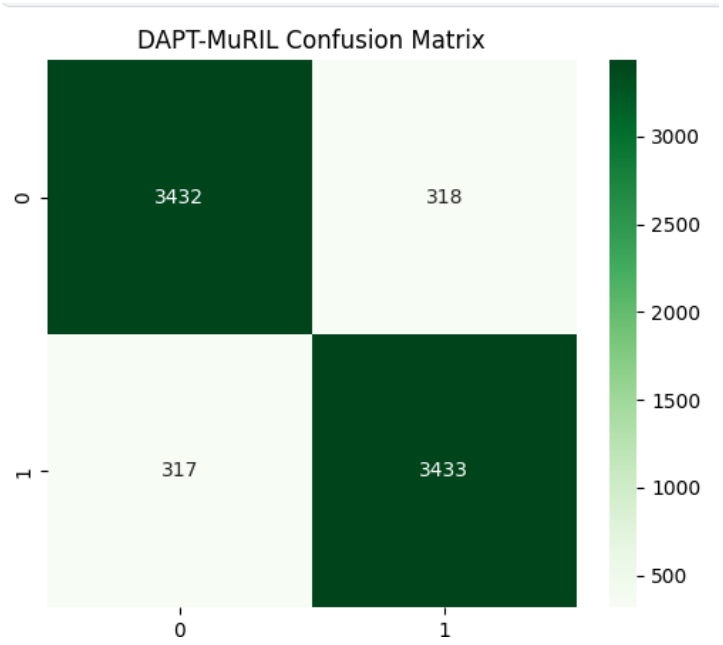


Figure 9.6: Confusion matrix of DAPT-MuRIL (0 = NOT, 1 = HOF)

Base MuRIL classifies 6,821 of the 7,500 comments correctly and DAPT-MuRIL 6,865, which is 44 more. Almost all of the improvement comes from fewer missed HOF comments: false negatives fall from 359 to 317, while false positives hardly change (320 to 318). The errors of DAPT-MuRIL are also more evenly spread (318 against 317), whereas base MuRIL misses more HOF comments than it wrongly flags.

9.4 Classification Report

Model	Class	Precision	Recall	F1-score	Support
MuRIL	NOT (0)	0.9052	0.9147	0.9099	3750
	HOF (1)	0.9138	0.9043	0.9090	3750
	Macro / weighted avg	0.9095	0.9095	0.9095	7500
DAPT-MuRIL	NOT (0)	0.9154	0.9152	0.9153	3750
	HOF (1)	0.9152	0.9155	0.9154	3750
	Macro / weighted avg	0.9153	0.9153	0.9153	7500

Table 9.5: Per-class results derived from the confusion matrices (they agree with the rounded scikit-learn reports in the notebook)

9.5 ROC Curve

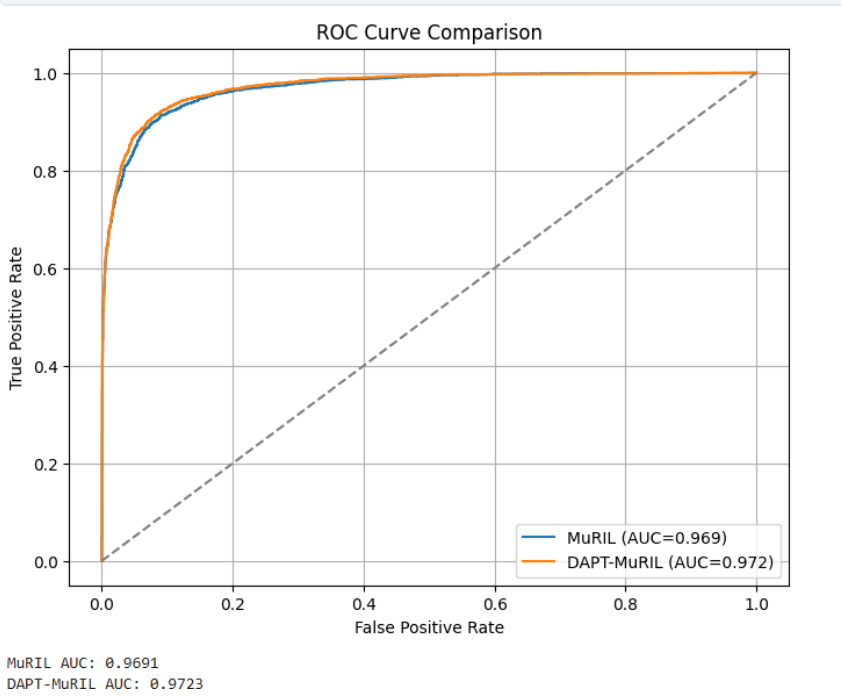


Figure 9.7: ROC curves of both models (AUC 0.9691 and 0.9723)

Both curves rise steeply and lie close together. The area under the curve is 0.9691 for base MuRIL and 0.9723 for DAPT-MuRIL, which means that either model ranks a randomly chosen HOF comment above a randomly chosen NOT comment in about 97% of cases. The difference of 0.003 is small.

9.6 Comparison: Base MuRIL vs DAPT-MuRIL

Model	Accuracy	Precision	Recall	F1
MuRIL	0.9095	0.9095	0.9095	0.9095
DAPT-MuRIL	0.9153	0.9153	0.9153	0.9153
Difference	+0.0059	+0.0059	+0.0059	+0.0059

Table 9.6: Comparison of base MuRIL and DAPT-MuRIL under identical fine-tuning settings

DAPT-MuRIL is better than base MuRIL on every measure in Tables 9.3 and 9.6, by about 0.6 percentage points in accuracy and F1. How much weight this difference deserves is discussed in Section 9.9.

9.7 Qualitative Test on Sample Sentences

Both models were also tried on four hand-written Marathi sentences (Table 9.7). The sentences are given here in transliteration with an English gloss; the original Devanagari inputs are in the notebook. Both models labeled all four as expected, but this is a very small check and does not replace the test-set results.

Sentence (transliterated)	English gloss	MuRIL	DAPT-MuRIL
Tu khup changla mitra ahes	You are a very good friend	NOT	NOT
Ha video khup chhan aahe	This video is very nice	NOT	NOT
Tujhyasarkhyanna baher kadhla pahije	People like you should be thrown out	HOF	HOF
Tu murkh ahes	You are a fool	HOF	HOF

Table 9.7: Predictions on four hand-written sample sentences

9.8 Computational Cost

Stage	Steps	Time
DAPT (1 epoch, 20,000 sentences)	625	668 s (about 11 min)
Fine-tuning base MuRIL (3 epochs)	2,814	3,536 s (about 59 min)
Fine-tuning DAPT-MuRIL (3 epochs)	2,814	3,548 s (about 59 min)
Test-set evaluation (per model)	235	about 2 min

Table 9.8: Run time of each stage on the dual-GPU Kaggle runtime

The adaptation stage adds about 11 minutes to a pipeline in which fine-tuning takes about an hour per model. The fine-tuning times of the two models are practically the same, as expected from identical settings.

9.9 Discussion

The DAPT-MuRIL classifier is better than base MuRIL on the 7,500-comment test set: accuracy 0.9153 against 0.9095, ROC-AUC 0.9723 against 0.9691 and evaluation loss 0.489 against 0.532. It is also ahead after every epoch, and its evaluation loss keeps falling while that of the base model does not. The gain, about 0.6 percentage points or 44 comments, comes mostly from fewer missed HOF comments. Precision and recall are practically equal for both classes in DAPT-MuRIL, so it treats the two classes evenhandedly. For moderation, roughly one comment in twelve is still misjudged, so human review remains necessary.

The difference is small, and several points limit what can be concluded from it.

- **Sampling error.** With 7,500 test comments, the standard error of an accuracy near 0.91 is about 0.33 percentage points. Treating the two accuracies as independent, the

difference of 0.59 points is only about 1.3 standard errors of the difference, which is not conclusive. A paired test on the per-comment predictions would be more precise.

- **Run-to-run variation.** An earlier execution of the same DAPT-MuRIL pipeline, before the baseline was added, gave an accuracy of 0.9107 and an AUC of 0.971, about half a point below the 0.9153 reported here. That is as large as the gap between the two models, so another run could shrink or remove the observed advantage.
- **Single run.** Each model was trained once. Repeating the experiment with several seeds would show how much of the gap is due to the adaptation.
- **Weak adaptation.** The adaptation used only 20,000 sentences for one epoch and its loss stayed around 9, so the adapted model changed only a little from the original, which makes a large effect unlikely.
- **Evaluation set.** The per-epoch evaluation used the test set, and there was no separate validation set. No checkpoint or hyperparameter was selected from these evaluations and the final epoch was used for both models, so the reported figures were not selected for.
- **Pooler initialization.** The adapted model starts with a randomly initialized pooler while the base model keeps the pretrained one, which is a disadvantage for the adapted model and so does not explain its advantage.
- **No text cleaning or duplicate check.** Duplicate comments could appear in both the training and test parts and raise both models' scores.

The results are therefore consistent with a small benefit from domain adaptation, but they do not prove it.

Chapter 10

Conclusion and Future Work

10.1 Conclusion

The project delivered a working pipeline for Marathi hate speech detection and a controlled comparison between base MuRIL and a domain adapted version. MuRIL was adapted by masked language modeling on a 20,000-sentence sample of MeCorpus, then both the original and the adapted model were fine-tuned on 30,000 MahaHate comments with identical settings and evaluated on 7,500 held-out comments. Base MuRIL reached an accuracy and F1-score of 0.9095 and a ROC-AUC of 0.9691. DAPT-MuRIL reached 0.9153 and 0.9723, with a lower evaluation loss and 44 fewer misclassified comments. The work covered dataset collection and analysis, data preparation, corpus creation, model loading, domain adaptive pretraining, fine-tuning, evaluation and documentation in a reproducible Kaggle notebook.

The gain from adaptation is small, comes from a single run and is of a size comparable to run-to-run variation, so it should be read as suggestive rather than proven. The other limits are the short adaptation whose loss stayed high, the use of the test set for per-epoch monitoring, the absence of text cleaning and duplicate checks, and the binary label scheme that groups hateful and offensive text. The system is suited to acting as a first screen that supports human moderators, not as a final judge of content.

10.2 Future Work

- Repeat both models with several seeds and apply a paired significance test to the predictions.
- Adapt for longer and on a larger corpus, including Dakshina and Marathi social media text, and check why the masked language modeling loss stayed high.
- Use a separate validation set, and check for duplicate texts across the splits.
- Add text cleaning and normalization and test its effect.
- Compare with IndicBERT, MahaBERT and mBERT.
- Predict the finer MahaHate classes to separate hateful, offensive and profane text.
- Investigate explainable AI techniques.
- Extend the system to multilingual and multimodal hate speech detection, and deploy it as a web application.

Bibliography

[1] S. Khanuja et al., “MuRIL: Multilingual Representations for Indian Languages,” arXiv:2103.10730, 2021.

[2] H. Patil et al., “L3Cube-MahaHate: A Tweet-based Marathi Hate Speech Detection Dataset and BERT models,” in *Proc. Third Workshop on Threat, Aggression and Cyberbullying (TRAC)*, 2022.

[3] B. Roark et al., “Processing South Asian languages written in the Latin script: the Dakshina dataset,” in *Proc. 12th Language Resources and Evaluation Conf. (LREC)*, 2020.

[4] J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proc. NAACL-HLT*, 2019.

[5] A. Vaswani et al., “Attention is all you need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[6] S. Gururangan et al., “Don’t stop pretraining: Adapt language models to domains and tasks,” in *Proc. ACL*, 2020.

[7] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *Proc. ICLR*, 2019.

[8] T. Wolf et al., “Transformers: State-of-the-art natural language processing,” in *Proc. EMNLP: System Demonstrations*, 2020.

[9] A. Paszke et al., “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[10] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.